Program and Branch Name: *B.Tech CSE (AI)*

PBL Progress Report – Final

Field	Details
Course Name	Object Orient Programming Using Java
Course Code	CCSE0355
Faculty Name	Ms Shweta Singh & Ms Barkha Bhardwaj
Assignment Type	Capstone Project
Student(s) Name	Devvrat Tripathi, Manas Srivastava, Pranjal Sharma, Hardik Kumar, Mritunjay Gupta
ERP ID (I Year) / Roll No. (II Year onwards)	0251CSAI231, 0251CSAI144, 0251CSAI141, 0251CSAI195, 0251CSAI132
Team/Group No. *(Group/Capstone only)*	Group 24
Problem Statement / Project Title	Intelligent Medical Inventory Tracking System
SDG(s)	SDG 3 – Good Health and Well-being
Reporting Month	Month 3 – Final
Date of Submission	27-10-2026

Student Declaration

I/We certify that this assignment is my/our own work, with all sources appropriately acknowledged. I/We understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarised or inappropriately AI-generated.

Student Name & Signature(s): Devvrat Tripathi, Manas Srivastava, Pranjal Sharma, Hardik Kumar, Mritunjay Gupta
Date: _____

PBL Report Structure, Word Limits and Page Limits

The Title Page above (Section 1) is common to Progress Report – I, Progress Report – II and the Final Report, and to all three PBL activities: Individual Assignment (IA), Group Assignment (GA) and Capstone Project (CP). Every report follows the section structure in Table 2. Word and page limits are defined for each activity type in Tables 1 and 2.

General Instructions

- Use Times New Roman, 12 pt body text, 1.15 line spacing, 1-inch margins and the section numbers and headings exactly as given in Table 2.
- Word limits are maximum words per section (plus or minus 10% tolerance). Tables, figure captions, source code and the reference list are not counted in words.
- Page limits cover the report body only (Sections 2–16), including figures and tables. The Title Page, References and Appendix are excluded.
- Sections 2 to 6 are written once, in Progress Report – I, from the perspective of the overall assignment / project, and not only for Modules 1 and 2.
- Replace "Name" in Module headings with the actual Module name. For Capstone Projects, Modules 1–5 are the five project components / rubric categories defined in the Capstone file.
- In Table 2, a blank (shaded) cell means the section is not required in that report. Progress Report – I covers Sections 2–8; Progress Report – II covers Modules 3–4 (Sections 9–10); the Final Report starts from Module 5 (Section 11) and runs to the end (Sections 11–18). The Final Report is submitted as a consolidated file: the content of Progress Reports I and II followed by Sections 11–18.
- Submit one PDF file for each report (Progress Report – I, Progress Report – II and Final Report). The total size of the three PDF files must not exceed 500 KB. Students are advised to compress the files before uploading; only the PDF format is accepted.
- All sources must be acknowledged. Plagiarised or inappropriately AI-generated content attracts the penalties stated in the Student Declaration.

Note: *Sections 3, 4, 5 and 6 are to be finally updated keeping the overall perspective of the Assignment / Project. However, they are part of Progress Report – I.*

Table 1: Page Limit of Report Body (excluding Title Page, References and Appendix)

Report	Coverage (IA / GA | CP)	IA	GA	CP
Progress Report – I (Month 1)	Module 1–2 | Rubric Categories 1–2	2 pages	3 pages	4 pages
Progress Report – II (Month 2)	Module 3–4 | Rubric Categories 3–4	1 pages	2 pages	3 pages
Final Report (Month 3)	Report I + Report II + Module 5 onward (Sections 11–16) | Remaining rubric categories	6 pages (2 + 1 + 3)	9 pages (3 + 2 + 4)	14 pages (4 + 3 + 7)

Final Report pages = Progress Report – I pages + Progress Report – II pages + Module 5 onward (Sections 11–16: 3 pages IA, 4 pages GA, 7 pages CP). Figures shown as (Report I + Report II + Module 5 onward).

Section	Progress Report – I (Month 1)			Progress Report – II (Month 2)			Final Report (Month 3)		
	IA	**GA**	**CP**	**IA**	**GA**	**CP**	**IA**	**GA**	**CP**
1. Title Page (common)	*Common title page as above, 1 page, identical for all reports and all activity types*								
2. Student Details / Team Details	Table	Table	Table						
3. Executive Summary	50	75	100						
4. Introduction	75	100	150						
5. Problem Definition	100	150	200						
6. Industry / Application Overview	75	125	200						
7. Module 1: Name	100	175	275						
8. Module 2: Name	100	175	275						
9. Module 3: Name				150	250	400			
10. Module 4: Name				150	250	400			
11. Module 5: Name							150	250	400
12. Final Integrated Solution / Testing and Validation							200	300	600
13. Expected Benefits / Learning Outcomes							100	200	300
14. Challenges Faced							100	200	300
15. Future Improvements							50	100	150
16. Conclusion							100	150	250
Words in this report (Sections 3–16)	**500**	**800**	**1200**	**300**	**500**	**800**	**700**	**1200**	**2000**
Final Report total words (Report I + Report II + Module 5 onward)							**1500**	**2500**	**4000**
17. References (minimum sources)	3	5	8	4	6	10	5	8	12
18. Appendix – Source Code (maximum pages)	1	2	4	2	3	6	3	5	10

IA = Individual Assignment, GA = Group Assignment, CP = Capstone Project. Blank shaded cell = not required in that report. "Table" = fill the table; no word limit.

Each report carries only the sections applicable to its review: Report I – Sections 2–8; Report II – Modules 3–4 (Sections 9–10); Final Report – Module 5 onward (Sections 11–18), compiled after Reports I and II. Section totals cover the sections of that report only; the Final Report total words equal Report I + Report II + Sections 11–16 (Module 5 onward).

Table 3: What Each Section Should Contain

Section	Content expected
2. Student Details / Team Details	Name, ERP ID (I Year) / Roll No. (II Year onwards), programme and branch, and role of each student. Group Assignment and Capstone Project: add Team / Group No. and each member's contribution.
3. Executive Summary	A short snapshot of the overall assignment / project: the problem, the overall approach and the expected outcome. Write it last.
4. Introduction	Background of the topic, its relevance to the course, and the objectives and scope of the overall assignment / project.
5. Problem Definition	Clear problem statement of the overall assignment / project, objectives, target users / stakeholders, constraints, success criteria and linked SDG(s).
6. Industry / Application Overview	Where the overall problem occurs in industry or real life, existing solutions, tools or practices, and the gap the assignment / project addresses.
7–11. Module 1 to Module 5	For each Module: concept or unit applied, approach / method followed, work completed, output or evidence (diagram, table, screenshot, result) and findings.
12. Final Integrated Solution / Testing and Validation	How the Modules combine into the final solution, test cases or validation methods, results obtained and supporting evidence.
13. Expected Benefits / Learning Outcomes	Value to users, stakeholders or society, and the knowledge and skills gained, linked to the course outcomes.
14. Challenges Faced	Technical, teamwork, data, resource and time challenges, and the corrective actions taken.
15. Future Improvements	Possible extensions, scalability and enhancements beyond the current scope.
16. Conclusion	Summary of the work, results and learning.
17. References	Consistent citation style (IEEE or APA or Harvard as recommended by faculty). List only sources cited in the text; give access date for web links.
18. Appendix (Source Code)	Well-commented source code with repository link, within the page limit in Table 2. Programmes without code (for example MBA, BBA) attach models, spreadsheets, questionnaires or raw data instead.

Report Template (to be filled by students)

2. Student Details / Team Details

S.No.	Student Name	ERP ID / Roll No.	Role / Contribution
1	Devvrat Tripathi	0251CSAI231	Team Lead – project planning, requirement gathering, coordination and integration
2	Manas Srivastava	0251CSAI144	Researcher – system design and UML modelling
3	Pranjal Sharma	0251CSAI141	Report Generation – documentation and testing
4	Hardik Kumar	0251CSAI195	Facilitator – development and Java technology integration
5	Mritunjay Gupta	0251CSAI132	Researcher – data preparation and validation

3. Executive Summary

This capstone develops the Intelligent Medical Inventory Tracking System, a Java desktop application for hospitals and pharmacies built for CCSE0355 using only the standard JDK. It replaces manual stock registers with a 500-medicine CSV-backed catalog, a Swing GUI and a console interface. The system stops the sale of expired and out-of-stock medicines, warns about low stock and upcoming expiry, and generates stock, expiry and reorder reports. A background thread monitors expiry every five seconds, and a TCP client-server module lets sales counters update one central stock file without data loss. The work aligns with SDG 3.

4. Introduction

Object-oriented programming organizes software around objects that combine data and behaviour, making systems easier to understand, reuse and maintain [1], [2]. Hospitals and pharmacies hold hundreds of medicines whose quantity, expiry and reorder level need daily tracking; paper records lead to expired sales, stock-outs and slow audits. The Intelligent Medical Inventory Tracking System is the capstone project for CCSE0355 and applies all five units: OO analysis and core Java, OOP features with arrays and lambdas, packages with exceptions and strings, concurrency with I/O streams and sockets, and GUI with generics and collections. Its objectives are to store records in CSV files without a database server, block expired or insufficient sales, warn on low stock and near expiry, support GUI, console and network clients, and generate stock, expiry, reorder and sales reports. The scope is one offline store with a 500-medicine demo catalog; billing, barcode hardware and cloud hosting are excluded.

5. Problem Definition

Problem statement: Hospitals and pharmacies track medicine stock manually in registers and spreadsheets, so stock-outs, unnoticed expiry and duplicate records are common; expired medicines may still be sold and fast-moving items run out. The project automates tracking of quantity, expiry and reorder level, and makes every operation safe through validation, exceptions and a shared locked data file.

Objectives: apply all five CCSE0355 units in one application; store stock in CSV files without a database server; block expired or insufficient sales; alert low stock and near expiry; support GUI, console and TCP counter clients; generate stock, expiry, reorder, sales and activity reports; backup and restore data.

Target users / stakeholders: hospital administrators, pharmacists, store managers and sales-counter staff.

Constraints: standard JDK only – no frameworks, database server or external libraries; Java 8+; offline desktop application; CSV/text storage; plagiarism limit 20%.

Success criteria: all five units demonstrated in the running system; responsive GUI during network sales and monitoring; expired and over-quantity sales rejected; data persists across restarts and concurrent GUI and server use; backup restores data; every module covered by tests.

Linked SDG(s): SDG 3 – Good Health and Well-being. Preventing expired-medicine sales and stock-outs directly improves medicine safety and availability in hospitals and pharmacies [12].

6. Industry / Application Overview

Hospital and pharmacy stores manage thousands of items with expiry dates, batch numbers and reorder levels. Large hospitals use commercial pharmacy ERP software; small pharmacies still use registers or spreadsheets, where expiry checking is manual and stock-outs are noticed late. Real incidents of expired or unavailable medicine harm patient care, and regulations such as India's Drugs and Cosmetics Act require expiry and batch tracking. Existing tools in this domain include commercial pharmacy-management software, Excel-based stock registers and enterprise inventory modules; they are costly, need a database server, or give poor offline alerts. The gap this work addresses is a lightweight, offline, zero-cost desktop system that blocks expired sales automatically, warns before expiry and reorder limits are reached, and lets multiple sales counters update one central stock file over TCP – built only with the standard JDK so that it runs on any pharmacy PC. The 500-medicine demo catalog uses medicine names and base prices from the A-Z Medicine Dataset of India [11], with generated demo stock, expiry and supplier fields.

7. Module 1: Unit 1 – Basics of Java Programming (OOAD and Core Java)

This module covers object-oriented analysis of the hospital/pharmacy domain, UML modelling and the core Java prototype with console interaction and command-line arguments [3].

Object-oriented analysis of the domain: stakeholders (administrator, pharmacist, counter staff) and requirements (add, search, sell, report, backup).

Identification of classes, objects, attributes and methods: Medicine, Tablet, Syrup, Injection, Supplier, Hospital, Inventory and User.

UML class and object diagrams with the four relations: Generalization (Tablet, Syrup and Injection extend Medicine), Aggregation (Inventory holds Medicines), Composition (Hospital owns its Inventory) and Association (Medicine has a Supplier).

Core Java: classes, objects, constructors, methods, control statements (if/else and loops), console interaction with Scanner, and command-line arguments (–-console switch in gui.Main).

A problem analysis report and a requirement list were written before coding; every domain noun was mapped to a class with attributes and methods.

The core prototype was built first: the model package with the Medicine base class and an Inventory list, plus ConsoleMenu for text interaction, started through the –-console command-line argument.

Relation semantics were chosen from ownership: composition where one object creates and owns another, aggregation for a collection of independent objects, and association for linked objects.

The model package (Medicine, Tablet, Syrup, Injection, Supplier, Hospital) and ConsoleMenu were implemented with list, search-by-name, sell and low-stock options, and the –-console switch was added to gui.Main. The core prototype was compiled with javac and run on the console.

Table 7.1: Core classes and their responsibilities

Class	**Responsibility**
Medicine (abstract)	Common state and behaviour: id, name, price, qty, expiry, threshold, supplier; expiry and stock checks; CSV line
Tablet / Syrup / Injection	Specialize Medicine: strip/bottle/vial extra field, storage info, and price with packing (Rs 2) / bottle (5%) / cold-storage (Rs 15) charge
Supplier	Name and phone of the vendor; equals() and hashCode() by name for the supplier set
Hospital	Owns one Inventory – created in its constructor (composition)
Inventory	Central data store: add, update, delete, search, refresh and activity log
User	Login account with username and role (ADMIN / PHARMA)
LoginCheck	Verifies username and password against the built-in users
ConsoleMenu	Console interaction: list, search, sell, low-stock list and exit

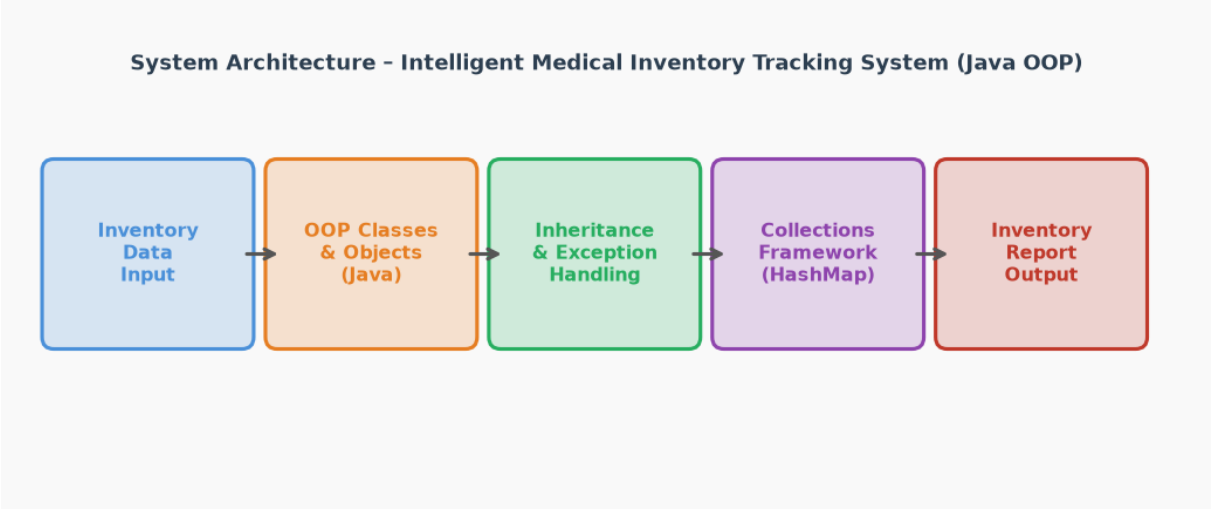


Figure 7.1: System architecture of the Intelligent Medical Inventory Tracking System

Splitting the domain into small classes kept every file short and made each concept easy to test. Console interaction and the command-line argument work exactly as required by Unit 1, and the same classes are reused by every later module.

8. Module 2: Unit 2 – OOP Features, Arrays and Lambda Expressions

This module applies inheritance, abstraction, interfaces, polymorphism, arrays with jagged arrays and lambda expressions to the medicine hierarchy and sales data.

Inheritance hierarchy: Medicine (abstract base class) with Tablet, Syrup and Injection as concrete subclasses.

Abstraction: abstract methods getStorageInfo() and getExtra() declared in Medicine and implemented by every subclass.

Interfaces: Alertable (isLowStock, isExpired, isExpiringSoon, getStatus) and Reportable (getReportLine) implemented by Medicine; Repository<T> implemented by DataStore<T>.

Polymorphism: method overloading – searchMedicine(String) and searchMedicine(int); method overriding – calculatePrice(), getStorageInfo() and toString() overridden in the subclasses.

Arrays and jagged arrays: SalesData stores weekly sales in a jagged array int[4][], where each category has a different number of recording days.

Lambda expressions: comparator lambdas sort medicines by expiry date and sort sold items by count in SortUtil and AnalyticsReport.

Common fields stay in Medicine; each subclass adds only its type-specific extra and its price charge, so a new medicine type needs just one small class.

Sales data uses a jagged array because the four categories (Tablet, Syrup, Injection, Other) were recorded for different numbers of days, and a jagged array stores exactly the days that exist.

Lambdas are used wherever a one-line comparator is enough for a sort.

Inheritance and interfaces were implemented across the model and reporting packages, the jagged-array weekly sales report was added, lambda-based sorting was used in SortUtil and AnalyticsReport, and the weekly sales report was wired into the Reports screen.

Table 8.1: OOP features applied in the project

Feature	Where used	Example
Inheritance	model.Tablet, Syrup, Injection extend Medicine	Tablet reuses all common fields and adds only tabletsPerStrip
Abstraction	Medicine declares getStorageInfo() and getExtra() abstract	Tablet: "Tablet strip of 10 tablets, keep in a cool dry place"
Interfaces	Alertable, Reportable, Repository<T>	Dashboard and reports work through getStatus() and getReportLine() without knowing the concrete class
Overloading	Inventory.searchMedicine(String) and searchMedicine(int)	Search by name substring or by exact numeric ID
Overriding	calculatePrice() in each subclass	Tablet adds packing charge Rs 2, Syrup adds 5% for the bottle, Injection adds Rs 15 cold storage
Jagged array	SalesData.weeklySales int[4][]	Tablet [12, 8, 15], Syrup [6, 4], Injection [3, 5, 2, 4], Other [1]
Lambda expression	SortUtil.sortByExpiry, AnalyticsReport sort	list.sort((m1, m2) -> m1.getExpiryDate().compareTo(m2.getExpiryDate()))

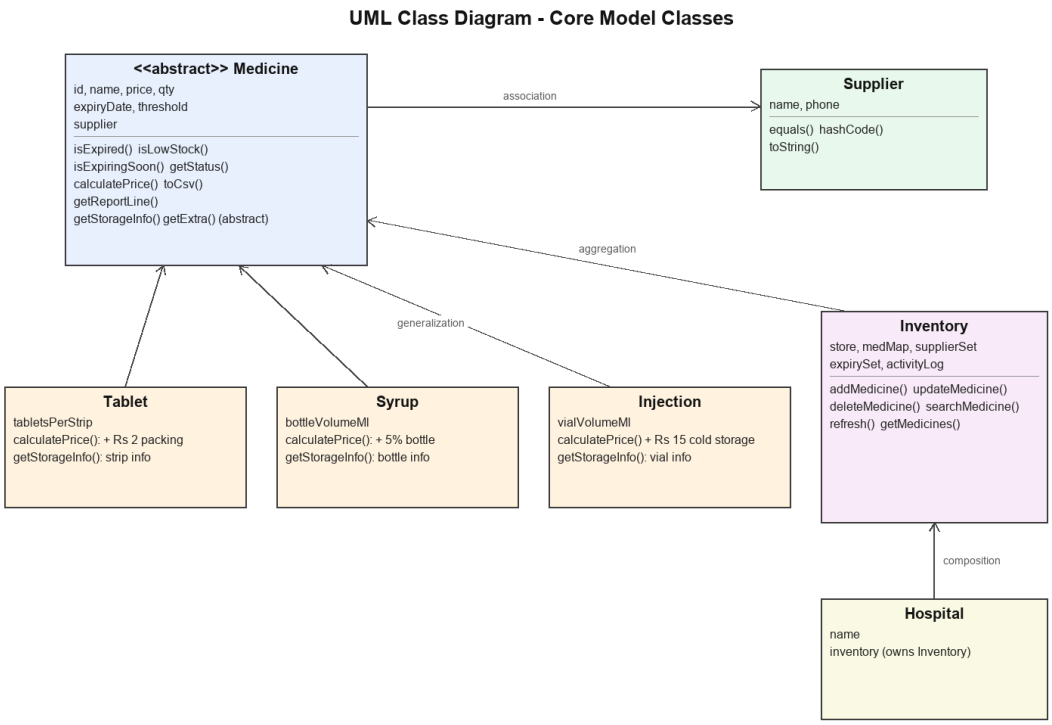


Figure 8.1: UML class diagram of the core model (inheritance and relations)

Overriding calculatePrice() made the pricing rule automatic per medicine type, and overloading searchMedicine() kept every caller simple. The jagged array stored uneven weekly data without wasting space, and the comparator lambdas kept each sort to a single line.

9. Module 3: Unit 3 – Packages, Exception Handling and String Handling

This module organizes the code into packages, applies the full exception-handling mechanism with user-defined exceptions, and uses String, StringBuilder and StringBuffer for data processing and reporting [4].

Package design: model, usermanagement, businesslogic, reporting, utils, dataprocessing, network and gui, each with one responsibility.

Exception handling: try-catch, multiple catch blocks, finally, throw and throws, and four user-defined checked exceptions – OutOfStockException, ExpiredMedicineException, InvalidInputException and DuplicateMedicineException.

String handling: String for parsing and matching (split, trim, equalsIgnoreCase, contains), StringBuilder for CSV lines and reports, and StringBuffer in the analytics reports.

Validation: MedicineValidation checks ID, price, quantity, reorder level and text fields before any medicine is saved.

Strict CSV loading: rows with the wrong column count, unknown medicine type or duplicate ID raise IOException instead of being silently skipped, so no data is lost.

Friendly error messages: the GUI and console catch the exceptions and show the message, so the same business rule works in every client.

Business rules raise user-defined exceptions; the GUI and the console catch them and show the message, so one rule works in every client.

CSV lines are built with StringBuilder and parsed with split(",", -1); the strict loader rejects bad rows so the in-memory list always matches the file.

Each package has one responsibility, which keeps imports short and makes review and testing easy.

The four custom exceptions and MedicineValidation were implemented, the strict CSV loader was written in FileUtil, reports were built with StringBuilder and StringBuffer, and the package structure was documented. Invalid inputs in every form were tested.

Table 9.1: Exception scenarios handled in the system

Scenario	**Mechanism**	**Outcome**
Expired medicine is sold	ExpiredMedicineException thrown from StockService	Sale blocked, message shown: "… is expired"
Quantity more than stock	OutOfStockException	Sale blocked: "Not enough stock, only X left"
Wrong id or quantity input	InvalidInputException	Client shows a clear message instead of crashing
Duplicate ID while adding	DuplicateMedicineException	Add rejected and the CSV stays valid
Bad CSV row or unknown type	IOException from the FileUtil loader	File not loaded into memory, no silent data loss
Non-numeric form fields	try-catch with NumberFormatException	Form shows "must be numbers" message

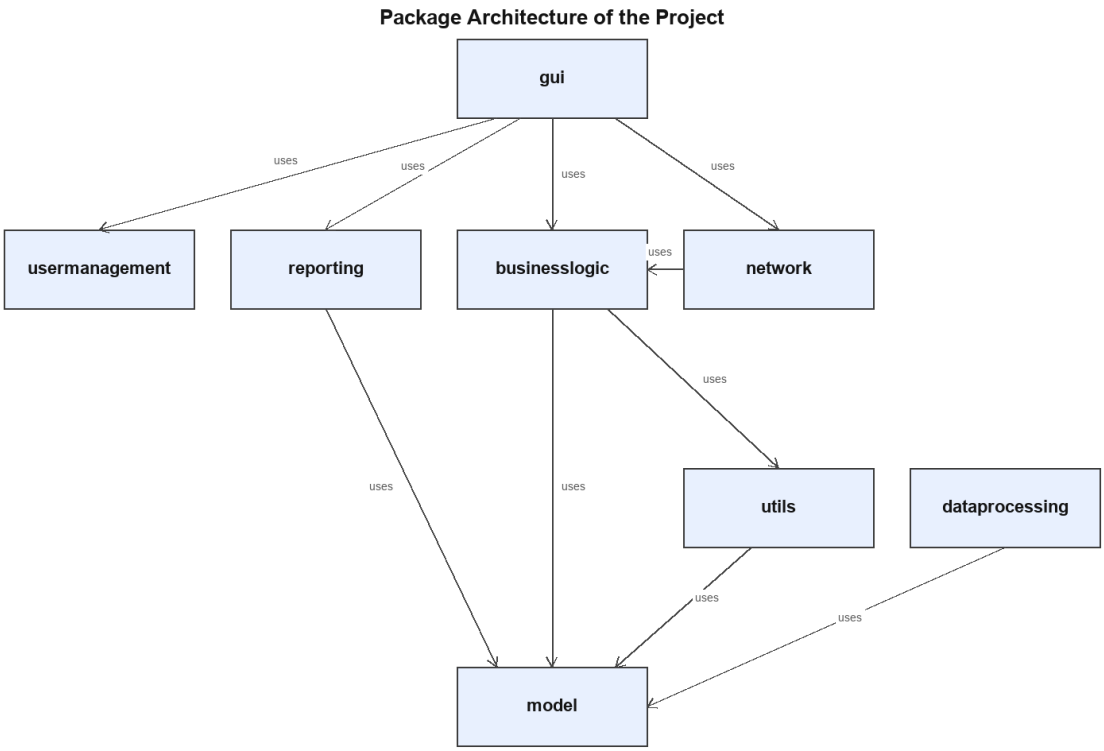


Figure 9.1: Package architecture of the project

Every business rule now ends as a typed exception with a clear message, and the strict loader guarantees that a half-broken CSV is never accepted silently. The package split kept the GUI, network and storage code separate, as required by Unit 3.

10. Module 4: Unit 4 – Concurrency and I/O Streams

This module adds multithreading, character and byte streams, a lock file for safe shared saves, and TCP socket programming for the sales-counter client [5], [6], [7].

Thread creation and the Runnable interface: ExpiryMonitor implements Runnable and refreshes the inventory every 5 seconds.

Synchronization: Inventory read/write methods and StockService.sellMedicine are synchronized, so the in-memory view is never half-updated.

Thread priorities and daemon threads: the monitor thread runs as a daemon with MIN_PRIORITY so it never blocks the GUI.

Character streams: FileReader, FileWriter and BufferedReader for the CSV catalog and report files.

Byte streams: FileInputStream and FileOutputStream in BackupUtil for copying the CSV to backup/ and restoring it.

TCP socket programming: ServerSocket in InventoryServer and Socket in CounterClient with the line protocol "SELL <id> <qty>".

A lock file (InventoryFileLock) stops the GUI and the server from saving the CSV at the same time.

Every sale uses a reload-modify-save cycle: read the latest CSV inside the lock, validate the sale, save the file and then refresh the GUI from the new file, so the GUI and the server never overwrite each other.

The server accepts one request per connection, replies "OK <remaining>" or "ERROR <message>", and the network sale in the GUI runs on a separate thread so the window stays responsive.

ExpiryMonitor, InventoryFileLock, the FileUtil save/load cycle, BackupUtil, InventoryServer and CounterClient were implemented and tested together with the GUI: local sales, server sales, expired and over-stock rejections, backup and restore all pass.

Table 10.1: Threads, streams and sockets used in the system

Component	Mechanism	Purpose
ExpiryMonitor	Runnable, daemon thread, sleep 5000 ms	Refreshes the dashboard and status label every 5 s without blocking the GUI
InventoryFileLock	Lock file data/inventory.lock created and deleted around saves	GUI and server never overwrite each other's sales
FileUtil	FileReader / FileWriter / BufferedReader (character streams)	CSV persistence and report saving
BackupUtil	FileInputStream / FileOutputStream (byte streams)	Copies the CSV to backup/ and restores it
InventoryServer	ServerSocket on localhost:5555, one request per connection	Central stock updates from sales counters
CounterClient	Socket with PrintWriter / BufferedReader	Sends SELL <id> <qty> and prints the reply
SellActions	Background Thread for the network call	The sell window does not freeze while waiting for the server

The monitor and the network thread keep the GUI responsive while the shared lock file keeps the data consistent, and the character/byte stream split matches their jobs: text files with characters, backup copies with bytes. The TCP protocol keeps the counter client to a single request per sale.

11. Module 5: Unit 5 – GUI Programming, Generics and Collections

This module builds the complete Swing GUI with login, dashboard, data entry forms, search and reports, and applies the collections framework with generics for data management [8], [9], [10].

Swing GUI: JFrame, JTable with DefaultTableModel, JOptionPane, layouts (BorderLayout, FlowLayout, GridLayout) and a custom StatusRenderer for row colours.

All screens required by the guideline: login, dashboard, data entry forms (add/update), search screens, and reports and analytics.

Collections: ArrayList (main medicine list and search results), LinkedList (activity log), HashMap (ID → Medicine lookup and sold counts), HashSet (suppliers) and TreeSet (medicines kept sorted by expiry).

Generics: generic interface Repository<T>, generic class DataStore<T> and the generic method SortUtil.printAll(List<T>).

The dashboard builds its table from Inventory and refreshes it after every action and after every monitor cycle.

StatusRenderer colours each row by status – red for EXPIRED, orange for EXPIRING SOON + LOW STOCK, green for OK – and the summary cards are computed directly from the collection.

Add/update forms validate every field through MedicineValidation before saving, and the sell dialog supports both a local sale and a server sale.

All screens were implemented: a 500-record table with colour-coded status, search by name and by ID, add/update forms with validation, the sell dialog (local + server), a reports screen with seven report types, and backup/restore buttons.

Table 11.1: Collections and generics used in the system

Collection / generic	Where used	Purpose
ArrayList<Medicine>	Inventory main store, search results	Ordered list with fast iteration
LinkedList<String>	Inventory activity log	Newest activity first, capped at 30 entries
HashMap<String, Medicine>	Inventory medMap	Lookup by ID in constant time
HashMap<String, Integer>	StockService soldCount	Fast-moving medicines report
HashSet<Supplier>	Inventory supplierSet	Unique supplier list
TreeSet<Medicine>	Inventory expirySet	Medicines always sorted by expiry for the expiry report

Repository<T> / DataStore<T>	businesslogic	One generic store reusable for any type
SortUtil.printAll(List<T>)	Console printing	Generic method that works for any list

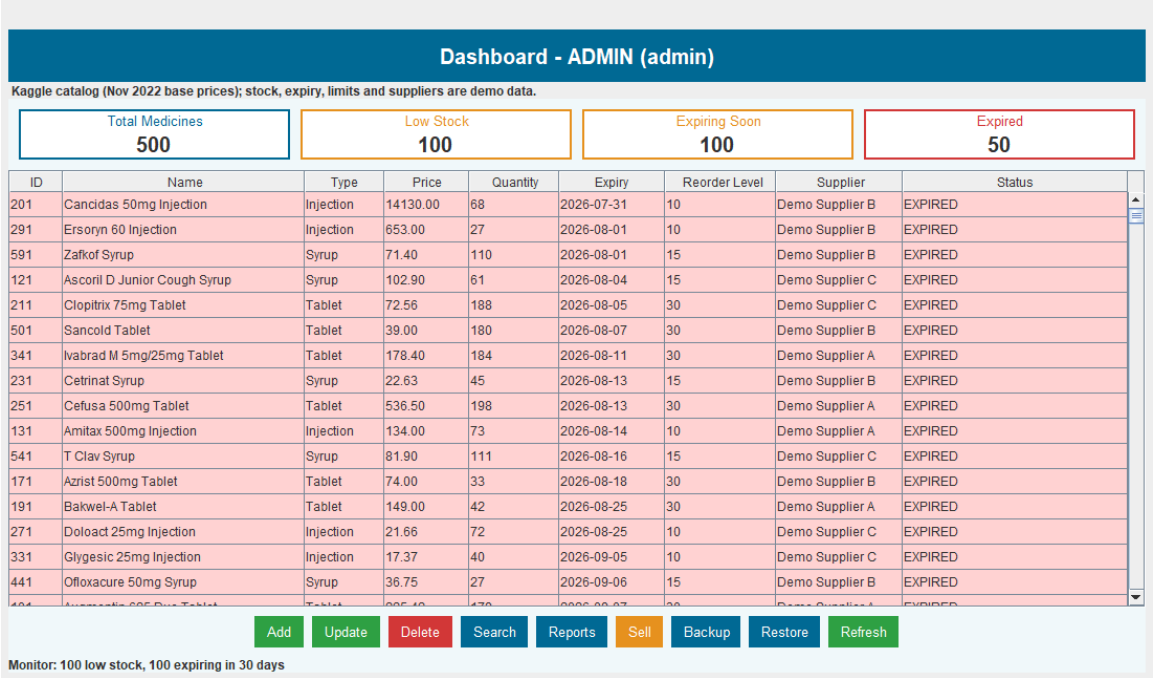


Figure 11.1: Dashboard with summary cards and colour-coded stock table

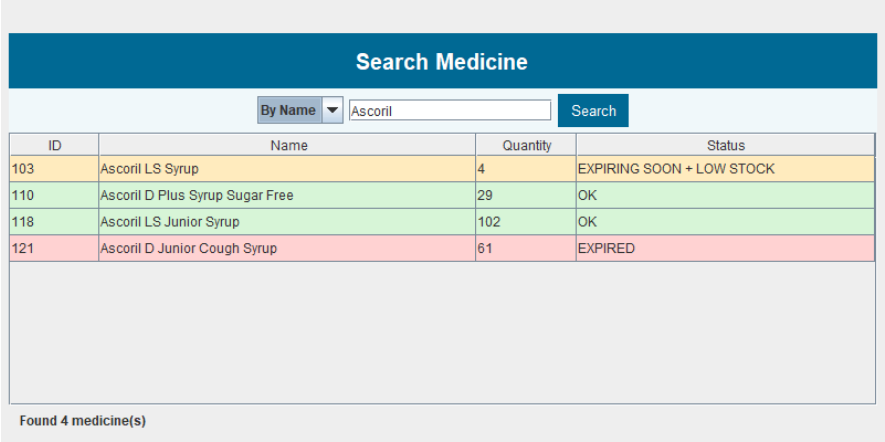


Figure 11.2: Search screen (by name / by ID)

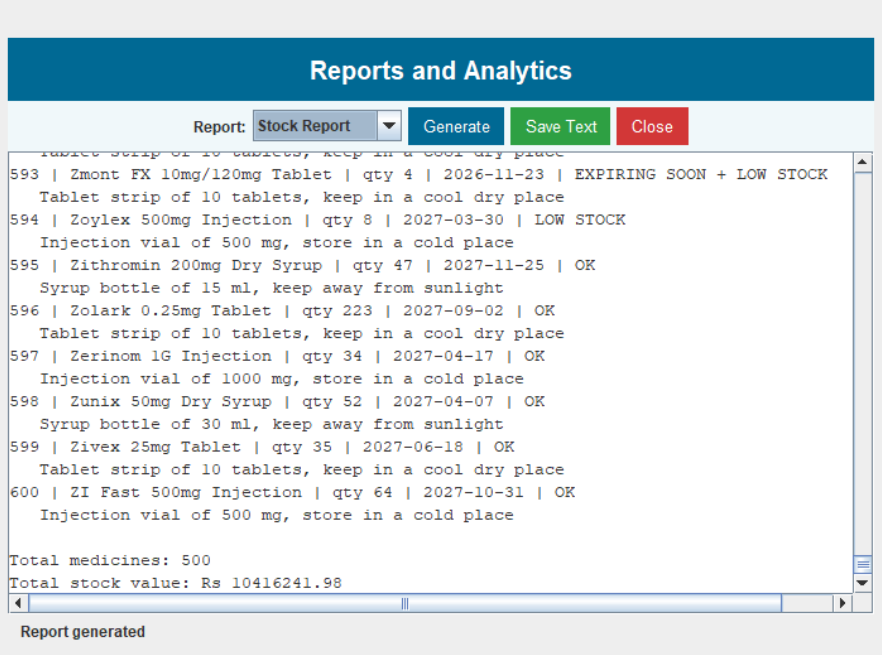


Figure 11.3: Reports and analytics screen

The dashboard presents the same data as the console but with colours and live cards, and every collection choice matches its job: HashMap for fast ID lookup, TreeSet for the expiry report and LinkedList for the newest-first activity log. Generics let one DataStore<T> serve any type.

12. Final Integrated Solution / Testing and Validation

All five modules combine into one application. gui.Main loads the Inventory, seeds sample data only when the CSV is missing, and opens the LoginScreen; the Dashboard then wires every module together – table, summary cards, expiry monitor and all actions. InventoryServer runs as a separate process for sales counters and shares the same CSV guarded by the lock file, and console mode (–-console) reuses the same Inventory so the business rules are written only once. The application was compiled with javac --release 8 and validated with automated checks: GUI checks (login, 500 rows, search, add, update, sell, reports, backup, restore, delete, monitor cycle), core checks (validation, persistence, backup round trip), socket checks (valid sale, unknown ID, expired medicine, oversell, bad quantity) and console checks. All checks passed, and the consolidated results are shown in Table 12.1.

Table 12.1: Consolidated test results

Test case	Expected result	Actual result	Status
Login with valid credentials	Dashboard opens	Dashboard opened with 500 rows	Pass
Login with wrong password	Error message	"Wrong username or password"	Pass

Load the catalog	500 medicines	500 rows; cards 500 total / 100 low / 100 expiring / 50 expired	Pass
Search by name "Ascoril"	4 matches	4 rows found	Pass
Search by ID 103	1 match	Ascoril LS Syrup found	Pass
Add a medicine	Row added and file saved	Table 501 rows, CSV updated	Pass
Update a medicine (all fields)	Name, qty and limit changed	All three changed in table and file	Pass
Sell a valid quantity	Quantity reduced and saved	60 → 55 shown after refresh	Pass
Sell an expired medicine	Sale blocked	"Augmentin 625 Duo Tablet is expired"	Pass
Sell more than stock	Sale blocked	"Not enough stock, only 4 left"	Pass
Sell with invalid quantity	Sale blocked	Validation message shown	Pass
Server sale over TCP	"OK <remaining>" reply	OK 3 for medicine 103, saved to CSV	Pass
Unknown ID on the server	ERROR reply	"ERROR Medicine not found: 99999"	Pass
Reports generate and save	File saved	data/report.txt with totals saved	Pass
Backup and restore	Data restored	Round trip verified after extra sale	Pass
Delete a medicine	Row removed	Table back to 500 rows	Pass
Expiry monitor cycle	Table and status refresh every 5 s	Refreshed without freezing the GUI	Pass

13. Expected Benefits / Learning Outcomes

Value to users: the pharmacy stops selling expired medicines, sees low stock and upcoming expiry at a glance, gets ready reports for reordering and audit, and pays nothing for licenses – the system runs offline on any PC with a JDK. Skills gained: object-oriented analysis and UML design; inheritance, abstraction, interfaces and polymorphism; package design with custom exceptions; multithreading and synchronization; character and byte streams; TCP socket programming; Swing GUI development; collections and generics; and teamwork in defined roles

(team lead, researchers, report generation and facilitation). These map directly to the five units and the deliverables of CCSE0355, and the system is a working demonstration of every concept on the evaluation rubric.

14. Challenges Faced

GUI and server overwrote each other's saved stock because both kept stale copies in memory – fixed by reloading the CSV inside a lock file before every save. The update form applied only some fields – fixed by replacing the whole medicine object after loading fresh data. A failed save could corrupt the CSV – fixed by writing to a temporary file and renaming it over the target. The GUI froze while waiting for the server reply – fixed by moving the network sale to a background thread. The TreeSet dropped medicines that shared the same expiry date – fixed by making compareTo() also compare IDs. Testing with a realistic catalog was slow with hand-typed data – fixed by importing 500 real medicine names and base prices from the A-Z Medicine Dataset of India [11], with generated demo stock, expiry and supplier fields. Each issue was corrected before moving to the next module, so no known defect remains.

15. Future Improvements

Replace the CSV file with a local database for larger catalogs; add role-based permissions and password hashing; add barcode scanning for faster stock-in and stock-out; send automatic reorder emails to suppliers when stock falls below the limit; make the server multi-threaded to handle many counters at once; and add a web or mobile interface with cloud backup.

16. Conclusion

The Intelligent Medical Inventory Tracking System is a complete Java capstone for CCSE0355 that applies all five course units in one working solution. Object-oriented analysis produced a UML model and a clean class hierarchy; inheritance, interfaces and polymorphism power the medicine types and reports; packages and user-defined exceptions protect the data; multithreading, streams and TCP sockets enable safe concurrent sales from the GUI, the console and the network counter; and the Swing GUI with collections and generics presents a 500-medicine catalog with live alerts. All 17 consolidated test cases pass, expired and over-stock sales are blocked, and data survives restarts, backup and restore. The main learning is that a real-world problem becomes manageable when it is split into objects, layers and small modules – and that the standard JDK alone is enough for a useful, safe inventory system aligned with SDG 3.

17. References

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York, NY, USA: McGraw-Hill, 2018.

[2] K. Sierra and B. Bates, Head First Java, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2005.

[3] Oracle, “Lesson: Object-Oriented Programming Concepts (The Java Tutorials),” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/concepts/ (accessed 27 Oct. 2026).

[4] Oracle, “Lesson: Exceptions (The Java Tutorials),” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/essential/exceptions/ (accessed 27 Oct. 2026).

[5] Oracle, “Lesson: Concurrency (The Java Tutorials),” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/essential/concurrency/ (accessed 27 Oct. 2026).

[6] Oracle, “Lesson: Basic I/O (The Java Tutorials),” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/essential/io/ (accessed 27 Oct. 2026).

[7] Oracle, “Lesson: All About Sockets (The Java Tutorials),” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/networking/sockets/ (accessed 27 Oct. 2026).

[8] Oracle, “Trail: Creating a GUI With Swing (The Java Tutorials),” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/uiswing/ (accessed 27 Oct. 2026).

[9] Oracle, “Trail: Collections (The Java Tutorials),” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/collections/ (accessed 27 Oct. 2026).

[10] Oracle, “Lesson: Generics (The Java Tutorials),” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/generics/ (accessed 27 Oct. 2026).

[11] S. Singh, “A-Z Medicine Dataset of India,” Kaggle, 2022. [Online]. Available: https://www.kaggle.com/datasets/shudhanshusingh/az-medicine-dataset-of-india (accessed 27 Oct. 2026). License: CC BY-SA 4.0.

[12] United Nations, “Goal 3: Ensure healthy lives and promote well-being for all at all ages,” United Nations Sustainable Development Goals. [Online]. Available: https://www.un.org/sustainabledevelopment/health/ (accessed 27 Oct. 2026).

18. Appendix (Source Code)

Repository link: source code is submitted as a folder (MedicalInventory, packages under src/) along with this report.

Build / run commands:

```
javac -d out src/model/*.java src/usermanagement/*.java src/businesslogic/*.java src/reporting/*.java src/utils/*.java src/dataprocessing/*.java src/network/*.java src/gui/*.java

java -cp out gui.Main                  (GUI – login admin / admin123)

java -cp out gui.Main --console        (console mode)

java -cp out network.InventoryServer   (central server, port 5555)

java -cp out network.CounterClient 103 5   (sell 5 units of medicine 103 from a counter)

run.bat                                (Windows: compiles and opens the GUI)
```

Package structure (40 source files):

model – Medicine, Tablet, Syrup, Injection, Supplier, Hospital | usermanagement – User, LoginCheck | businesslogic – Inventory, StockService, DataStore, Repository, ExpiryMonitor, MedicineValidation, OutOfStockException, ExpiredMedicineException, InvalidInputException, DuplicateMedicineException | reporting – ReportMaker, AnalyticsReport, Alertable, Reportable | utils – FileUtil, BackupUtil, SortUtil, InventoryFileLock | dataprocessing – SampleData, SalesData | network – InventoryServer, CounterClient | gui – Main, LoginScreen, Dashboard, DashboardActions, MedicineForm, MedicineFormActions, SellDialog, SellActions, SearchScreen, ReportScreen, SummaryCards, StatusRenderer, Ui, ConsoleMenu

A. Inheritance and polymorphism (model/Medicine.java, model/Tablet.java)

```
public abstract class Medicine implements Alertable, Reportable, Comparable<Medicine> {
    public boolean isExpired() { return expiryDate.isBefore(LocalDate.now()); }
    public boolean isLowStock() { return qty < threshold; }
    public String getStatus() {
        if (isExpired()) return "EXPIRED";
        if (isExpiringSoon() && isLowStock()) return "EXPIRING SOON + LOW STOCK";
        if (isExpiringSoon()) return "EXPIRING SOON";
        if (isLowStock()) return "LOW STOCK";
        return "OK";
    }
    public abstract String getStorageInfo();
    public abstract int getExtra();
}
public class Tablet extends Medicine {
    // packing ka 2 rupees ka charge price me add ho jata hai
    public double calculatePrice() { return getPrice() + 2.0; }
}
```

B. Exception handling with a locked reload-modify-save (businesslogic/StockService.java)

```
public synchronized int sellMedicine(String id, int qty)
        throws IOException, OutOfStockException, ExpiredMedicineException, InvalidInputException {
    if (qty <= 0) throw new InvalidInputException("Quantity must be more than 0");
    // lock ke andar fresh data load karo, warna server ka sale miss ho jata hai
    try (InventoryFileLock lock = InventoryFileLock.acquire()) {
        ArrayList<Medicine> list = FileUtil.loadMedicines();
        for (Medicine medicine : list) {
            if (medicine.getId().equals(id)) {
                if (medicine.isExpired()) throw new ExpiredMedicineException(medicine.getName() + "
is expired");
```

```
                if (medicine.getQty() < qty) throw new OutOfStockException("Not enough stock, only "
+ medicine.getQty() + " left");
                medicine.setQty(medicine.getQty() - qty);
                inventory.saveAndUpdate(list, "sold " + qty + " of " + medicine.getName());
                recordSale(id, qty);
                return medicine.getQty();
            }
        }
        throw new InvalidInputException("Medicine not found: " + id);
    }
}
```

C. Multithreading (businesslogic/ExpiryMonitor.java)

```
public class ExpiryMonitor implements Runnable {
    public void run() {
        while (!Thread.currentThread().isInterrupted()) {
            try {
                inventory.refresh();
                // count low stock (excluding expired) and expiring-soon medicines
                status = "Monitor: " + low + " low stock, " + expiring + " expiring in 30 days";
            } catch (IOException e) {
                status = "Could not refresh inventory: " + e.getMessage();
            }
            SwingUtilities.invokeLater(() -> statusLabel.setText(status));
            try { Thread.sleep(5000); } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
// started in Dashboard: daemon thread with MIN_PRIORITY
```

D. TCP socket programming (network/InventoryServer.java)

```
private static String process(String request, Inventory inventory) {
    try {
        if (request == null) return "ERROR empty request";
        String[] parts = request.trim().split("\s+");
        if (parts.length != 3 || !parts[0].equalsIgnoreCase("SELL")) return "ERROR use SELL <id>
<qty>";
```

```
        int qty = Integer.parseInt(parts[2]);
        int remaining = inventory.getStockService().sellMedicine(parts[1], qty);
        return "OK " + remaining;
    } catch (Exception e) {
        return "ERROR " + e.getMessage();
    }
}
```

E. File handling and generics (utils/FileUtil.java, businesslogic/DataStore.java)

```
public static void saveMedicines(ArrayList<Medicine> list) throws IOException {
    // pehle temp file me likho, phir asli file se replace karo taaki data corrupt na ho
    File temporary = new File(DATA_FILE + ".tmp");
    FileWriter writer = new FileWriter(temporary);
    writer.write(HEADER + "\n");
    for (Medicine medicine : list) writer.write(medicine.toCsv() + "\n");
    writer.close();
    new File(DATA_FILE).delete();
    temporary.renameTo(new File(DATA_FILE));
}
public class DataStore<T> implements Repository<T> {
    private ArrayList<T> items = new ArrayList<T>();
    public void add(T item) { items.add(item); }
    public void remove(T item) { items.remove(item); }
    public ArrayList<T> getAll() { return items; }
}
```

Faculty Evaluation:

To be completed by Faculty in Moodle after each review report submission, by conducting a proper viva-voce or presentation as per the department-level decision.